CE 310

Python Tutorial — Monte Carlo Simulation

Week 13 · Session 2 · Hands-on Walkthrough

University of Arizona · Dr. Wooyoung Jung · Fall 2026

Today's Plan — No CSV Required

Every number today is generated by the code. The model is the one Week 12 ranked; what changes is that the inputs become **distributions** instead of low–high pairs.

By the end of this walkthrough you will know how to:

1. Turn a plausible range into a distribution — and see what that assumes
2. Draw correlated-free samples from **one** seeded stream
3. Read **percentiles** and **exceedance probabilities** off a simulation
4. Decompose output variance by input
5. Convert a distribution into a **contingency percentage**

File needed: none.

The Model, Unchanged

$$\text{item cost} = Q \times P \times (1 + \text{overhead}) \times (1 + \text{escalation})$$

```python
import numpy as np
import matplotlib.pyplot as plt

Q0, P0, OH0, ESC0 = 2000, 21.2636, 0.13, 0.05
RMSE = 0.2418          # the price model's residual spread, log10 units
N    = 10_000

def cost(q, p, oh, esc):
    return q * p * (1 + oh) * (1 + esc)

BASE = cost(Q0, P0, OH0, ESC0)
print(f'base cost = ${BASE:,.0f}')
```

Expected: $50,459 — the same base cost as Week 12.

Step 1 — A Range Is Not a Distribution

Week 12 gave each input a low and a high. Monte Carlo needs a shape. Read each range as a **90% interval**, so the half-width is 1.645 σ.

```
Z90 = 1.645

SIG_Q    = Q0 * 0.15 / Z90      # +/-15% take-off       -> 182.4 CY
SIG_OH   = 0.05 / Z90           # 8% to 18%             -> 0.0304
SIG_ESC  = 0.03 / Z90           # 2% to 8%              -> 0.0182
SIG_LOGP = 0.05                 # three quotes in hand -> x1.12 / x0.89
```

⚠️ `SIG_LOGP` is a *choice*, not a measurement. Week 12 used ±1 RMSE — the spread of the whole market — because it had nothing better. An estimator with quotes does. Step 7 puts the market spread back.

Step 2 — One Seed, Drawn in Order

```
np.random.seed(42)
q    = np.random.normal(Q0, SIG_Q, N)
p    = 10 ** np.random.normal(np.log10(P0), SIG_LOGP, N)
oh   = np.random.normal(OH0, SIG_OH, N)
esc  = np.random.normal(ESC0, SIG_ESC, N)

item_cost = cost(q, p, oh, esc)
```

> 💡 Seeding **once** gives you one long stream, handed out in the order you ask. Seed before each draw and all four inputs receive the *same* numbers — perfectly correlated, which is the opposite of what you want and quietly wrong.

Step 2 (cont.) — Why the Price Line Is Different

```
p = 10 ** np.random.normal(np.log10(P0), SIG_LOGP, N)
```

The model predicts **$\log_{10}$ price**, so the uncertainty is Normal *in log space*.
Exponentiating gives a **log-normal** price: never negative, and skewed right.

```
print(f'price mean   ${p.mean():.2f}')
print(f'price median ${np.median(p):.2f}')
```

Expected: mean slightly above the median. That gap is the whole reason the mean item cost lands above the base cost, and it grows with `SIG_LOGP`.

Step 3 — The Analytical Check

Cost is **linear** in quantity, so the standard deviation of a quantity-only simulation is exact arithmetic — no simulation required.

```
np.random.seed(42)
cost_qty = cost(np.random.normal(Q0, SIG_Q, N), P0, OH0, ESC0)

sim   = cost_qty.std()
exact = SIG_Q * P0 * (1 + OH0) * (1 + ESC0)
print(f'simulated  ${sim:,.0f}')
print(f'analytical ${exact:,.0f}')
```

Expected: $4,617 against $4,601 — a $16 gap on $4,600, which is sampling noise. Do not "fix" it.

Step 4 — Percentiles and Exceedance

Two lines, and they are what an estimator actually uses.

```
p10, p50, p90 = np.percentile(item_cost, [10, 50, 90])
prob_over = (item_cost > 60_000).mean()

print(f'P10 ${p10:,.0f}   P50 ${p50:,.0f}   P90 ${p90:,.0f}')
print(f'P(cost > $60,000) = {prob_over:.4f}')
```

> 💡 `(array > threshold)` is an array of `True` and `False`; `.mean()` of it is the *fraction* that are True. That is the whole exceedance-probability idiom.

Expected: P10 $41,427 · P50 $50,350 · P90 $61,068 · P = 0.1215

Step 5 — Variance Decomposition

Run the simulation once per input, with only that input random. Because the inputs are independent, the four variances add.

```
def var_from(which):
    np.random.seed(42)
    a = dict(q=Q0, p=P0, oh=OH0, esc=ESC0)
    if which == 'q':   a['q']   = np.random.normal(Q0, SIG_Q, N)
    if which == 'p':   a['p']   = 10 ** np.random.normal(np.log10(P0), SIG_LOGP, N)
    if which == 'oh':  a['oh']  = np.random.normal(OH0, SIG_OH, N)
    if which == 'esc': a['esc'] = np.random.normal(ESC0, SIG_ESC, N)
    return cost(**a).var()
```

Note this function re-seeds on purpose — each run varies **one** input, so a shared stream cannot correlate anything.

Step 5 (cont.) — Reading the Shares

```
v = {k: var_from(k) for k in ('p', 'q', 'oh', 'esc')}
total = sum(v.values())
for k in v:
    print(f'{k:<4} {100 * v[k] / total:5.2f}%')
```

Expected: price 59.15% · quantity 36.37% · overhead 3.16% · escalation 1.32%

> ⚠️ Same **order** as Week 12's tornado, different **proportions**. Variance squares the swings, so a decomposition compresses ratios that a swing ranking spreads out. The two are not interchangeable.

Step 6 — From a Distribution to a Contingency

A contingency is a probability statement, which is why Week 12 could not produce one. Search upward until the overrun risk drops below 5%.

```
for pct in range(0, 105, 5):
    prob_over = (item_cost > BASE * (1 + pct / 100)).mean()
    print(f'{pct:>3}%  ->  P(overrun) = {prob_over:.4f}')
    if prob_over < 0.05:
        break
```

Expected: 30% — bid the item at about $65,600. The overrun probability there is 0.037, so about 1 job in 27 still comes in above it.

Step 7 — What the Answer Rests On

Change one number — the price spread — from quotes back to the whole market.

```
np.random.seed(42)
q_m   = np.random.normal(Q0, SIG_Q, N)
p_m   = 10 ** np.random.normal(np.log10(P0), RMSE, N)   # the only change
oh_m  = np.random.normal(OH0, SIG_OH, N)
esc_m = np.random.normal(ESC0, SIG_ESC, N)

cost_market = cost(q_m, p_m, oh_m, esc_m)
print(f'std with quotes    ${item_cost.std():,.0f}')
print(f'std without quotes ${cost_market.std():,.0f}')
```

Expected: $7,716 against $36,594 — a factor of **4.74**.

Quick Check — Before You Start the In-Class Exercise

Reproduce these, then open `Wk13.03_In_Class_Exercise.ipynb`

Quantity	Expected
base cost	$50,459
mean / std, full MC	$50,844 / $7,716
P10 / P50 / P90	$41,427 / $50,350 / $61,068
variance shares	59.1 / 36.4 / 3.2 / 1.3 %
contingency for 5% risk	30%
std ratio, no quotes	4.74×

Question: the 80% band is $19,641 — 38.9% of base. Week 12's all-corner spread was 170.3%. Same model, same inputs. Where did the other 131 points go?

Quick Reference — Sampling

Task	Code
Seed once	`np.random.seed(42)`
Normal draws	`np.random.normal(mu, sigma, N)`

Task	Code
Log-normal price	`10 ** np.random.normal(np.log10(P0), s, N)`
Fixed input	leave it as a scalar

Quick Reference — Reading a Simulation

Task	Code
Percentiles	`np.percentile(x, [10, 50, 90])`
Exceedance	`(x > threshold).mean()`

Task	Code
Variance	`x.var()`
Histogram	`ax.hist(x, bins=60)`

Before You Start

- [ ] Every expected value above reproduced
- [ ] `np.random.seed(42)` called **once** before the four draws
- [ ] Price drawn as `10 ** normal(log10(P0), sigma)`, not `normal(P0, sigma)`
- [ ] You can say in one sentence why the mean cost exceeds the base cost

Then open **`Wk13.03_In_Class_Exercise.ipynb`** and begin Section A.

Before You Submit

⚠️ Following this submission format exactly is essential — with a class this size, grading depends on it. Submissions that don't comply will be penalized.

1. **Run all cells first.** In Colab: Runtime → Run all. Wait until every cell shows a checkmark — no ⏳ spinners remaining.
2. **Download as .ipynb.** File → Download → Download .ipynb. Do not submit PDF or .py.
3. **Do not edit the `print` lines.** Lines like `print(f"ANSWER_A1 = {ANSWER_A1}")` must remain exactly as written — changing or deleting them means that part of your answer will not be counted.

Before You Submit (continued)

4. **Do not change `SEED = 42`.** Your simulation results must use this exact seed or they will not match the expected values.
5. **File name — `CE310_W13_<NetID>.ipynb`**, e.g., `CE310_W13_abc123.ipynb`.
6. **Upload** the `.ipynb` file to D2L Dropbox by **9:00 AM next Wednesday**.